

Aspect	Sealed Classes (sealed, permits)
Java Version	Java 17
Year	2021
Feature Status	Final / Standard feature
UniqueClassName	SealedClasses
Purpose	Restrict which classes or interfaces are allowed to extend/implement a particular type
Main Keywords	sealed, permits, final, non-sealed
Primary Benefit	Controlled inheritance and better domain modeling
Best Suited For	Closed type hierarchies, domain models, state machines, API design, expression trees

Example

```
public sealed interface Payment
    permits CreditCardPayment, UPIPayment, CashPayment {
}
```

```
public final class CreditCardPayment implements Payment {
}
```

```
public final class UPIPayment implements Payment {
}
```

```
public final class CashPayment implements Payment {
}
```

Here, Payment explicitly controls its permitted implementations. A class such as CryptoPayment cannot implement Payment unless it is added to the permitted hierarchy.

Category	Advantages of Sealed Classes	Disadvantages of Sealed Classes
Inheritance control	Restricts inheritance to explicitly permitted types	Makes the hierarchy less extensible
Design	Clearly communicates the intended class hierarchy	Requires additional design decisions

Type safety	Prevents unexpected subclasses from being introduced	Can be inconvenient when third-party extension is required
Maintainability	Makes closed domain models easier to understand	Adding a new subtype requires modifying the sealed hierarchy
Pattern matching	Works very well with pattern matching and exhaustive switch	Benefits are reduced when code does not use pattern matching
API design	Library authors can control supported implementations	Consumers cannot freely extend the API
Security	Helps prevent unauthorized or unintended inheritance	It is not a replacement for authentication or security controls
Refactoring	Compiler can identify many hierarchy-related problems	Large sealed hierarchies can become cumbersome
Documentation	permits documents the allowed hierarchy directly in source code	The hierarchy can become verbose
Domain modeling	Excellent for representing a fixed set of business concepts	Poor fit for domains expected to evolve through external extensions

When to Use

Scenario	Should You Use?	Why	Real-Time Example
Fixed business categories	✓ Yes	Number of valid types is controlled	Payment → CashPayment, CardPayment, UPIPayment
State machines	✓ Yes	States can be explicitly restricted	OrderState → Created, Paid, Shipped, Delivered
Result types	✓ Yes	Clearly defines possible outcomes	Result → Success, Failure

Expression trees	✓ Yes	Useful when all expression types are known	Expression → Add, Multiply, Constant
API response models	✓ Yes	Prevents unsupported response implementations	ApiResponse → SuccessResponse, ErrorResponse
Domain-driven design	✓ Yes	Represents a controlled domain vocabulary	Employee → PermanentEmployee, ContractEmployee
Security-sensitive hierarchy	□ Sometimes	Can prevent unintended extension	Restricted framework types
Framework extension points	✗ Usually no	Framework users may need to create subclasses	Plugin architecture
Open-ended plugin systems	✗ No	New implementations are expected from external developers	Plugin interface
Frequently changing hierarchy	✗ Usually no	Every new subtype requires hierarchy changes	Rapidly evolving product categories
Simple inheritance	✗ Usually no	Sealed classes add unnecessary constraints	Basic Vehicle → Car, Bike model

Real-Time Use Cases

Use Case	Sealed Hierarchy	Example
Payment processing	Payment	CreditCardPayment, UPIPayment, CashPayment
Order management	OrderState	Created, Confirmed, Shipped, Delivered, Cancelled
Banking transactions	Transaction	Deposit, Withdrawal, Transfer
API results	Result	Success, Failure

User authentication	AuthResult	Authenticated, Unauthenticated, Locked
Notifications	Notification	Email, SMS, Push
Financial instruments	FinancialProduct	Loan, FixedDeposit, SavingsAccount
Compiler/AST design	Expression	BinaryExpression, Literal, Variable

Example: Order State

```
public sealed interface OrderState
    permits Created, Paid, Shipped, Delivered, Cancelled {
}
```

```
public final class Created implements OrderState {
}
```

```
public final class Paid implements OrderState {
}
```

```
public final class Shipped implements OrderState {
}
```

```
public final class Delivered implements OrderState {
}
```

```
public final class Cancelled implements OrderState {
}
```

The compiler and developers now have an explicit understanding that these are the intended OrderState implementations.

final, sealed, and non-sealed

Modifier	Meaning	Can Be Extended?	Typical Usage
final	Completely prevents inheritance	✗ No	Leaf class
sealed	Allows only specified subclasses	✓ Only permitted types	Controlled hierarchy
non-sealed	Reopens inheritance	✓ Yes	Extensible branch of a

	below a sealed type		sealed hierarchy
--	---------------------	--	------------------

Example:

```
public sealed class Payment
    permits CardPayment, CashPayment {
}

public final class CardPayment extends Payment {
}

public non-sealed class CashPayment extends Payment {
}

// Now other classes can extend CashPayment
public class RetailCashPayment extends CashPayment {
}
```

When Not to Use Sealed Classes

Situation	Why Avoid It	Better Choice
You expect external developers to extend the type	sealed prevents arbitrary implementations	interface or normal class
New implementations are frequently added	Every addition affects the hierarchy	Normal interface
Building a plugin architecture	Plugins need independent implementations	interface
Framework extension points	Users often need customization	abstract class / interface
The hierarchy has only one implementation	Sealing provides little value	final class
The type is merely used for polymorphism	Restriction may be unnecessary	Normal interface/abstract class

Alternate Feature

Sealed Classes	Alternative	When Alternative Is Better
sealed interface	Normal interface	When implementations should be open-ended
sealed class	abstract class	When subclasses need shared state/behavior but should remain extensible
sealed class	final class	When there should be exactly one implementation and no inheritance
sealed interface	Enum	When the set of possible values is small, fixed, and behavior is simple
Sealed hierarchy	Composition	When inheritance is not actually required
Sealed hierarchy + pattern matching	Traditional polymorphism	When each subtype can encapsulate its own behavior

Sealed Classes vs Alternatives

Feature	Sealed Class	Normal Interface	Abstract Class	Enum	Final Class
Controls inheritance	✓ Strongly	✗ No	✗ No	✓ Fixed	✓ Completely
Multiple implementations	✓ Yes	✓ Yes	✓ Yes	Limited	✗ No
External extension	✗ Restricted	✓ Yes	✓ Yes	✗ No	✗ No
Shared implementation	✓ Yes	Default methods	✓ Excellent	✓ Yes	✓ Yes
Fixed set of types	✓ Excellent	✗	✗	✓ Excellent	✗
Pattern matching friendly	✓ Excellent	Good	Good	✓ Excellent	Good

Best for domain modeling	★★★★★	★★★	★★★★	★★★★	★★
Best for plugin architecture	✗	★★★★★	★★★★	✗	✗

Bottom Line

Question	Answer
What problem does it solve?	It gives developers controlled inheritance .
Biggest advantage?	The compiler knows the permitted type hierarchy.
Biggest disadvantage?	It restricts future extensibility.
Use when?	Your domain has a known and intentionally closed set of subtypes .
Don't use when?	Third parties or future modules are expected to freely add implementations.
Best alternative?	Usually a normal interface for an open hierarchy, or an abstract class when shared state/behavior is needed.
Most important Java 17 combination?	Sealed classes + pattern matching for switch provides particularly strong support for exhaustive type handling.

Java 17 — Strong Encapsulation

Aspect	Strong Encapsulation
Java Version	17
Year	2021
Feature Status	Final / Standard behavior

UniqueClassName	StrongEncapsulation
Introduced	Java 9 with JPMS, strengthened in Java 17
Main Purpose	Prevent applications from accessing internal JDK implementation APIs through reflective or other unsupported mechanisms
Related Feature	Java Platform Module System (JPMS)
Important Java 17 Change	JDK internal elements are strongly encapsulated by default; the previous ability to use --illegal-access for broad relaxation was removed

What Is Strong Encapsulation?

Concept	Explanation
Encapsulation	Hides implementation details and exposes only intended APIs
Strong encapsulation	Goes further by preventing unauthorized access to internal JDK packages/classes
Example	Application should use java.util public APIs instead of directly accessing JDK internal implementation classes
Why Java 17?	Java 17 made strong encapsulation of JDK internals the default without the old broad --illegal-access escape hatch
Main Goal	Better reliability, maintainability, security, and freedom for the JDK to evolve internally

Advantages and Disadvantages

Advantages	Disadvantages
Protects JDK implementation details	Older applications may fail after upgrading to Java 17
Prevents accidental dependence on internal APIs	Some reflection-heavy libraries may require changes

Improves application maintainability	Migration can require identifying illegal reflective access
Gives the JDK freedom to change internal implementations	Legacy frameworks may encounter <code>InaccessibleObjectException</code>
Encourages use of supported public APIs	Developers may need to find replacement APIs
Improves modularity	Module configuration can become more complex
Reduces fragile dependencies on JDK internals	Temporary workarounds using <code>--add-opens</code> may still be needed in some cases
Helps improve platform security	Debugging migration problems can initially be difficult

Example — Accessing a JDK Internal API

✗ Avoid

// Example of depending on a JDK internal implementation
import sun.misc.Unsafe;

```
public class InternalApiExample {
    public static void main(String[] args) {
        // Direct dependency on an internal JDK API
    }
}
```

`sun.misc.Unsafe` and other internal APIs are not intended to be general-purpose application APIs.

✓ Prefer a Supported API

```
import java.util.concurrent.atomic.AtomicInteger;

public class SupportedApiExample {

    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger(0);

        counter.incrementAndGet();

        System.out.println(counter.get());
    }
}
```

```
}  
}
```

The application depends on a supported Java API rather than a JDK implementation detail.

Reflection Example

Strong encapsulation becomes particularly noticeable with reflection.

Situation	Java 17 Behavior
Accessing your own public class	✓ Normally allowed
Accessing a public JDK API	✓ Allowed
Accessing an internal JDK implementation package	✗ May be blocked
Calling <code>setAccessible(true)</code> against strongly encapsulated JDK internals	✗ Can result in <code>InaccessibleObjectException</code>
Accessing a package explicitly opened by its module	✓ Possible

For example, code attempting deep reflection into JDK internals may encounter:

```
java.lang.reflect.InaccessibleObjectException
```

This is one of the most visible effects of strong encapsulation when migrating older applications to Java 17.

When to Use Strong Encapsulation

Strong encapsulation isn't normally something you "turn on" in application code. In Java 17, it is a **platform/module-system behavior** that you should design your applications to respect.

Scenario	Use?	Reason	Real-Time Example
New Java 17 application	✓ Yes	Follow supported APIs from the beginning	Spring Boot application
Enterprise application	✓ Yes	Removes dependencies on JDK	Java 8 → Java 17 migration

migration		internals	
Modular application	✓ Yes	Clearly defines module boundaries	module-info.java
Security-sensitive application	✓ Yes	Limits access to implementation details	Banking application
Large microservice platform	✓ Yes	Reduces hidden platform dependencies	Hundreds of Java services
Library development	✓ Yes	Prevents consumers from depending on implementation details	Internal Java library
Reflection-heavy framework	□ Carefully	May require explicit module opens	Dependency injection framework
Legacy application	□ Carefully	Existing internal API usage may break	Old Java EE application
Application using JDK internals	✗ Avoid	Internal APIs aren't stable contracts	sun.* dependencies

Real-Time Use Cases

Use Case	How Strong Encapsulation Helps	Example
Banking application	Prevents application code from relying on JDK implementation details	Payment processing service
Microservices	Reduces accidental dependencies on internal JDK classes	REST service running on Java 17
Framework development	Forces frameworks toward supported APIs/module boundaries	Dependency injection framework

Cloud applications	Makes applications less dependent on a particular JDK implementation	Containerized Java service
Java migration	Identifies obsolete internal API dependencies	Java 8 → Java 17
Security-sensitive systems	Restricts deep access to platform internals	Authentication service
Library development	Encourages stable public API usage	Enterprise shared library
JDK evolution	Allows JDK maintainers to change internals without preserving them as public contracts	GC implementation changes

Strong Encapsulation and JPMS

Strong encapsulation is closely related to the **Java Platform Module System (JPMS)**.

```
module com.example.payment {
    requires java.sql;

    exports com.example.payment.api;
}
```

JPMS Keyword	Purpose
module	Defines a module
requires	Declares module dependencies
exports	Makes packages available to other modules
opens	Allows reflective access to a package
opens ... to	Allows reflective access to specific modules
exports vs opens	exports is primarily for normal access; opens is for deep reflection

exports vs opens

Declaration	Normal Access	Deep Reflection
No declaration	✗	✗
exports com.example.api	✓	Limited/not equivalent to opens
opens com.example.internal	✗ for ordinary cross-module access	✓
opens com.example.internal to framework	✗ for ordinary cross-module access	✓ to specified module

This distinction is especially important for frameworks that use reflection.

When Not to Fight Strong Encapsulation

Situation	What You Should Do
Your library accesses sun.* APIs	Find a supported replacement
Reflection fails against a JDK internal class	Change the implementation
An old framework requires deep reflection	Upgrade the framework if possible
A dependency breaks on Java 17	Upgrade or replace the dependency
You are considering --add-opens everywhere	Treat it as a migration workaround, not the preferred architecture
You control the application's own modules	Properly design exports/opens boundaries

Alternate Features / Approaches

Strong Encapsulation	Alternative / Related	When to Prefer the Alternative
----------------------	-----------------------	--------------------------------

	Approach	
Strongly encapsulated JDK internals	Public Java SE APIs	Almost always; preferred solution
JPMS module boundaries	Traditional package-level encapsulation	Small applications that don't need JPMS
opens for reflection	Explicit framework configuration	When only specific reflective access is required
Internal API dependency	Standard replacement API	When migrating legacy code
--add-opens workaround	Upgrade/refactor the dependency	Preferred for production applications
Module boundaries	private / protected / package-private	For ordinary class-level encapsulation
JDK internal API	Third-party supported library	When Java SE doesn't provide the required functionality

Strong Encapsulation vs Alternatives

Feature	Strong Encapsulation	Java Access Modifiers	JPMS	Public API Design
Primary scope	JDK/platform internals	Classes/members	Modules/packages	Application/library API
Prevents internal JDK access	✓	✗	✓	Indirectly
Controls class members	✗	✓	✗	✓
Controls module boundaries	✓	✗	✓	✓
Supports reflection	✓	Limited	✓	Depends

control				
Best for	Platform integrity	Class design	Large applications	Stable APIs
Java 17 relevance	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★

Migration Tip: Java 8 → Java 17

Problem	Recommended Solution
Application uses JDK internal API	Replace it with supported Java API
Library uses illegal reflection	Upgrade the library
Framework needs reflective access to your module	Configure opens appropriately
Third-party dependency is incompatible	Upgrade/replace dependency
Temporary migration blocker	Consider a narrowly scoped --add-opens only when necessary
Long-term production solution	Remove dependency on inaccessible internals

Bottom Line

Question	Answer
What is Strong Encapsulation?	A stronger boundary preventing unauthorized access to JDK implementation internals.
Why important in Java 17?	Java 17 made strong encapsulation the default and removed the broad --illegal-access mechanism.
Biggest advantage?	Applications rely on stable public APIs rather than fragile implementation details.
Biggest disadvantage?	Legacy reflection-heavy applications and libraries can break during Java 17 migration.

When to use?	Essentially always respect it in Java 17 applications.
When not to use?	You generally don't disable it; instead, fix dependencies on internal APIs.
Preferred alternative?	Use supported Java SE APIs , appropriate access modifiers, and well-designed JPMS boundaries.
Real-world importance	Particularly significant when migrating Java 8/11 applications and older frameworks to Java 17 .